

LAPORAN PRAKTIKUM 2
PEMROGRAMAN BERORIENTASI OBJEK
RELASI ANTAR OBJEK



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Nurfiah, S. ST, M. Kom

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

A. Langkah Kerja Praktikum dan Penjelasan Source Code

1. Membuat Package Baru dan Class Transaksi

Pertama, buat package bernama Pekan2. Selanjutnya, membuat class baru bernama Transaksi.java. Class ini digunakan untuk menyimpan informasi setiap transaksi yang dilakukan oleh rekening.

Class Transaksi memiliki tiga atribut:

- idTransaksi untuk menyimpan ID transaksi.
- jenis untuk menyimpan jenis transaksi, yaitu Kredit untuk setor tunai dan Debit untuk tarik tunai.
- nominal untuk menyimpan jumlah uang yang ditransaksikan.

Sesuai dengan modul, class Transaksi juga memiliki method cetakDetail() yang digunakan untuk menampilkan 1 baris detail transaksi.

```
package Pekan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp " + nominal);
    }
}
```

Pada constructor, nilai id, jenis, dan nominal digunakan untuk mengisi atribut objek Transaksi. Method cetakDetail() kemudian digunakan untuk menampilkan data transaksi dalam satu baris. Dengan adanya class ini, setiap transaksi setor maupun tarik dapat dibuat sebagai sebuah objek tersendiri.

2. Membuat Relasi Rekening dengan Transaksi

Setelah class Transaksi dibuat, class Rekening diperbarui dengan menambahkan `ArrayList<Transaksi>`. Hal ini digunakan untuk menerapkan hubungan one-to-many, yaitu satu rekening dapat memiliki banyak transaksi, karena berdasarkan modul, objek Transaksi akan disimpan di dalam objek Rekening setiap kali terjadi aktivitas penyetoran atau penarikan tunai.

```
// Implementasi Asosiasi (1-to-many)
```

```
ArrayList<Transaksi> riwayatTransaksi;
```

Kemudian `ArrayList` diinisialisasi pada constructor agar tidak `NullPointerException`.

```
this.riwayatTransaksi = new ArrayList<>();
```

`riwayatTransaksi` digunakan sebagai tempat untuk menyimpan seluruh objek Transaksi yang dilakukan oleh suatu rekening. Inisialisasi dilakukan menggunakan `this.riwayatTransaksi = new ArrayList<>();` agar `ArrayList` sudah siap digunakan ketika objek Rekening dibuat.

3. Mencatat Transaksi Setor Tunai

Method `setorTunai()` kemudian diperbarui agar tidak hanya mengubah saldo, tetapi juga membuat objek Transaksi ketika setor tunai berhasil.

```
public void setorTunai(double nominal) {
    if (nominal > 10000) {
        saldo += nominal;
        // Merekam riwayat (Pembuatan Objek Transaksi di
dalam memori
        String idTrx = "TRX-S-" +
System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp " +
formatRupiah(nominal) + " berhasil. Saldo saat ini: Rp " +
formatRupiah(saldo));
    } else {
        System.out.println("Gagal: Nominal setor harus lebih
dari 10.000!");
    }
}
```

Jika nominal setor lebih dari Rp10.000, saldo akan ditambahkan terlebih dahulu. Setelah transaksi berhasil, program membuat objek Transaksi baru. `String idTrx = "TRX-S-" + System.currentTimeMillis();` → digunakan untuk membuat ID transaksi setor. Lalu, `Transaksi trxBaru = new`

Transaksi(idTrx, "Kredit", nominal); → untuk membuat objek transaksi dengan jenis Kredit. Nantinya, objek tersebut dimasukkan ke dalam riwayat menggunakan riwayatTransaksi.add(trxBaru);. Dengan begitu, transaksi setor yang berhasil akan tersimpan di dalam rekening.

4. Mencatat Transaksi Tarik Tunai

Method tarikTunai() juga digunakan untuk mencatat transaksi penarikan ke dalam riwayatTransaksi.

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp " + formatRupiah(saldo));
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Transaksi sebesar Rp " + formatRupiah(nominal) + " berhasil! Saldo anda tersisa: Rp " + formatRupiah(saldo));
    }
}
```

Method ini memiliki beberapa kondisi. Jika nominal < Rp10.000, transaksi akan ditolak. Jika nominal > saldo, transaksi juga ditolak. Jika penarikan berhasil, saldo dikurangi dan dibuat objek Transaksi dengan jenis Debit. Kemudian objek transaksi disimpan menggunakan riwayatTransaksi.add(trxBaru);. Jadi setiap transaksi tarik tunai yang berhasil juga masuk ke dalam riwayat rekening.

5. Menambahkan Fitur Menampilkan Tiga Transaksi Terbaru

Selain fitur utama praktikum, program ini juga ditambahkan fitur untuk menampilkan tiga transaksi terbaru apabila jumlah transaksi sudah lebih dari tiga (berdasarkan challenge 3). Fitur ini dibuat sebagai tambahan agar data transaksi yang ditampilkan tidak terlalu banyak ketika riwayat transaksi sudah bertambah.

```
public void cetakTigaTransaksiTerbaru() {
    if (riwayatTransaksi.size() <= 3) {
```

```

        System.out.println("Transaksi belum lebih dari 3!");
        return;
    }
    System.out.println("--- 3 Transaksi Terbaru ---");
    int jumlah = riwayatTransaksi.size();

    for (int i = jumlah - 1; i >= jumlah - 3; i--) {
        riwayatTransaksi.get(i).cetakDetail();
    }
    System.out.println("-----");
}

```

Program terlebih dahulu mengecek jumlah transaksi, jika jumlah transaksi belum lebih dari tiga, program akan menampilkan pesan bahwa transaksi belum lebih dari tiga. Jika jumlah transaksi lebih dari tiga, program mengambil data mulai dari indeks terakhir menggunakan

```
for (int i = jumlah - 1; i >= jumlah - 3; i--)
```

Perulangan dimulai dari transaksi paling akhir sehingga transaksi terbaru ditampilkan paling atas. Setiap transaksi kemudian ditampilkan menggunakan method `riwayatTransaksi.get(i).cetakDetail()`. Dengan cara itu, hanya tiga transaksi terakhir yang ditampilkan dan urutannya dimulai dari transaksi yang paling baru.

6. Menghubungkan Fitur Tambahan dengan Main.java

Setelah fitur tambahan pada `Rekening.java` selesai dibuat, fitur tersebut kemudian dihubungkan dengan menu pada `Main.java`. Menu yang digunakan adalah `System.out.println("6. Cetak 3 Transaksi Terbaru");`. Kemudian dibuat case 6.

```

case 6:
    if(akunAktif == null) {
        System.out.println("Error: Anda belum
membuka rekening");
    } else {
        akunAktif.cetakTigaTransaksiTerbaru();
    }
    break;

```

Sebelum menampilkan transaksi, program mengecek apakah sudah terdapat akun yang aktif. Jika `akunAktif` masih null, program akan menampilkan pesan bahwa rekening belum dibuka. Jika sudah ada akun aktif, program akan memanggil method `cetakTigaTransaksiTerbaru()` untuk menampilkan tiga transaksi terbaru.

7. Struktur Program Main.java

Pada Main.java, program tetap menggunakan ArrayList<Rekening> untuk menyimpan beberapa rekening.

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();  
Rekening akunAktif = null;
```

daftarRekening digunakan untuk menyimpan kumpulan objek rekening, sedangkan akunAktif digunakan untuk menunjukkan rekening yang sedang digunakan. Program juga menggunakan Scanner untuk menerima input dari pengguna dan while-loop agar menu dapat dijalankan berulang kali.

Struktur menu yang digunakan adalah:

```
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti / Pilih Akun Aktif  
6. Cetak 3 Transaksi Terbaru  
0. Keluar  
[Akun Aktif: Belum ada]  
Pilih Menu:
```

Dengan struktur tersebut, pengguna dapat melakukan transaksi dan melihat riwayat transaksi.

8. Hasil Pengujian

a. Pengujian fitur utama praktikum

Pada pengujian ini dilakukan transaksi setor dan tarik tunai. Setelah transaksi berhasil, saldo rekening berubah sesuai nominal transaksi dan data transaksi disimpan ke dalam riwayatTransaksi.

1) Membuat rekening

```
Pilih Menu: 1  
Masukkan No Rekening: 123456  
Masukkan Nama Pemilik: Amiratul  
Masukkan Saldo Awal: 1500000  
Rekening atas nama Amiratul berhasil dibuat dengan saldo Rp 1.500.000,00
```

Program berhasil membuat rekening baru dan menetapkan rekening tersebut sebagai akun aktif.

2) Setor Tunai

```
[Akun Aktif: Amiratul (123456)]  
Pilih Menu: 2  
Masukkan nominal setor: 250000  
Setor tunai Rp 250.000,00 berhasil. Saldo saat ini: Rp 1.750.000,00
```

Setelah setor tunai berhasil, program membuat objek Transaksi baru dengan jenis Kredit. Objek tersebut kemudian ditambahkan ke dalam riwayatTransaksi menggunakan method add().

3) Tarik Tunai

```
[Akun Aktif: Amiratul (123456)]
Pilih Menu: 3
Masukkan nominal tarik tunai: 550000
Transaksi sebesar Rp 550.000,00 berhasil! Saldo anda tersisa: Rp 1.200.000,00
```

Setelah tarik tunai berhasil, program membuat objek Transaksi baru dengan jenis Debit. Objek tersebut kemudian ditambahkan ke dalam riwayatTransaksi menggunakan method add().

b. Pengujian fitur tambahan

Setelah melakukan lebih dari tiga transaksi, pilih menu “7. Cetak 3 Transaksi Terbaru” untuk melihat tiga transaksi terbaru.

```
Pilih Menu: 6
--- 3 Transaksi Terbaru ---
ID: TRX-S-1790169771715 | Jenis: Kredit | Nominal: Rp 452000.0
ID: TRX-T-1790169747478 | Jenis: Debit | Nominal: Rp 100000.0
ID: TRX-T-1790169084575 | Jenis: Debit | Nominal: Rp 550000.0
-----
```

Dari gambar tersebut, dapat dilihat bahwa fitur tambahan berhasil menampilkan tiga transaksi terbaru dengan transaksi yang paling baru berada di bagian paling atas.

9. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat dipahami bahwa hubungan antar objek pada Java dapat diterapkan menggunakan asosiasi one-to-many. Pada program ini, satu objek Rekening dapat memiliki banyak objek Transaksi yang disimpan menggunakan ArrayList. Setiap transaksi setor dan tarik tunai yang berhasil akan dibuat sebagai objek Transaksi dan disimpan ke dalam riwayat rekening. Selain itu, program dikembangkan dengan fitur tambahan untuk menampilkan tiga transaksi terbaru, sehingga transaksi dapat ditampilkan mulai dari yang paling baru. Dengan praktikum ini, konsep class, object, constructor, method, ArrayList, serta hubungan antar objek dapat diterapkan secara langsung pada program perbankan sederhana.

B. Tugas/Latihan Praktikum

1. Integrasi Rekam Jejak Penarikan

Pada tugas ini, method tarikTunai() diperbarui agar transaksi penarikan yang berhasil juga dicatat ke dalam riwayatTransaksi. Setiap transaksi penarikan dibuat sebagai objek Transaksi dengan jenis Debit dan ID yang menggunakan prefiks TRX-T-. Pencatatan hanya dilakukan apabila saldo mencukupi dan penarikan berhasil.

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal
nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak
mencukupi. Saldo Anda: Rp " + formatRupiah(saldo));
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" +
System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit",
nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Transaksi sebesar Rp " +
formatRupiah(nominal) + " berhasil! Saldo anda tersisa: Rp " +
formatRupiah(saldo));
    }
}
```

Pada bagian else, program terlebih dahulu mengurangi saldo sesuai nominal penarikan. Setelah penarikan berhasil, program membuat objek Transaksi dengan jenis Debit dan ID yang diawali TRX-T-. Objek tersebut kemudian ditambahkan ke riwayatTransaksi menggunakan method add(). Karena proses pencatatan berada di dalam kondisi transaksi berhasil, penarikan yang gagal tidak akan masuk ke dalam riwayat transaksi.

- Hasil Pengujian

Pengujian dilakukan dengan melakukan tarik tunai menggunakan nominal yang memenuhi ketentuan dan saldo yang mencukupi. Setelah transaksi berhasil, saldo berkurang dan data penarikan dicatat sebagai transaksi Debit.

```
[Akun Aktif: Amiratul (123456)]
Pilih Menu: 3
Masukkan nominal tarik tunai: 750000
Transaksi sebesar Rp 750.000,00 berhasil! Saldo anda tersisa: Rp 842.000,00
```

2. Fitur Cetak Mutasi Rekening

Pada tugas ini dibuat method cetakMutasi() untuk menampilkan seluruh riwayat transaksi pada rekening. Method menggunakan perulangan foreach untuk mengambil setiap objek Transaksi yang tersimpan di dalam riwayatTransaksi. Jika belum terdapat transaksi, program akan menampilkan pesan “Belum ada transaksi pada rekening ini”.

Pada langkah praktikum sebelumnya, menu 6 digunakan untuk fitur tambahan menampilkan tiga transaksi terbaru. Namun, pada tugas 2 ditentukan bahwa fitur cetakMutasi() harus dihubungkan melalui menu 6. Oleh karena itu, fitur tambahan tiga transaksi terbaru dipindahkan dari menu 6 menjadi menu 7, sedangkan menu 6 digunakan untuk fitur cetak mutasi sesuai instruksi tugas.

```
System.out.println("6. Cetak Mutasi (Riwayat)");  
System.out.println("7. Cetak 3 Transaksi Terbaru");
```

Membuat method cetakMutasi()

```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening ini");  
    } else {  
        System.out.println("--- Riwayat Transaksi ---");  
        for (Transaksi trx : riwayatTransaksi) {  
            trx.cetakDetail();  
        }  
        System.out.println("-----");  
    }  
}
```

Program terlebih dahulu memeriksa apakah riwayatTransaksi kosong menggunakan method isEmpty(). Jika tidak terdapat transaksi, program menampilkan “Belum ada transaksi pada rekening ini”. Jika terdapat transaksi, program menggunakan perulangan for (Transaksi trx : riwayatTransaksi) untuk mengambil setiap objek Transaksi yang tersimpan. Selanjutnya, method cetakDetail() dipanggil untuk menampilkan ID, jenis, dan nominal setiap transaksi.

Menambahkan menu cetak mutasi pada main.java

Agar method cetakMutasi() dapat digunakan melalui program utama, ditambahkan menu baru:

```
System.out.println("6. Cetak Mutasi (Riwayat)");
```

Kemudian ditambahkan case 6:

```
case 6:
if (akunAktif == null) {
System.out.println("Error: Anda belum membuka rekening");
} else {
akunAktif.cetakMutasi();
}
break;
```

Program terlebih dahulu memeriksa apakah terdapat akun aktif. Jika akun aktif tersedia, program akan memanggil method cetakMutasi() untuk menampilkan seluruh riwayat transaksi pada akun tersebut. Penambahan menu ini sesuai dengan instruksi Tugas 2 pada modul.

- Hasil Pengujian

Pengujian dilakukan setelah rekening melakukan beberapa transaksi setor dan tarik tunai. Ketika menu 6. Cetak Mutasi (Riwayat) dipilih, program menampilkan seluruh transaksi yang telah tersimpan pada rekening.

```
[Akun Aktif: Amiratul (123456)]
Pilih Menu: 6
--- Riwayat Transaksi ---
ID: TRX-S-1790170851824 | Jenis: Kredit | Nominal: Rp 50000.0
ID: TRX-T-1790170870949 | Jenis: Debit | Nominal: Rp 750000.0
ID: TRX-S-1790171550768 | Jenis: Kredit | Nominal: Rp 78000.0
ID: TRX-S-1790171574456 | Jenis: Kredit | Nominal: Rp 5500000.0
ID: TRX-T-1790171597671 | Jenis: Debit | Nominal: Rp 2550000.0
ID: TRX-T-1790171614859 | Jenis: Debit | Nominal: Rp 75000.0
-----
```

3. Kesimpulan

Berdasarkan tugas yang telah dilakukan, dapat dipahami bahwa setiap transaksi yang berhasil dilakukan dapat dicatat sebagai objek Transaksi dan disimpan ke dalam riwayatTransaksi. Method cetakMutasi() juga dapat digunakan untuk menampilkan seluruh transaksi yang tersimpan pada rekening. Dengan demikian, program perbankan yang dibuat tidak hanya dapat melakukan setor dan tarik tunai, tetapi juga dapat menyimpan serta menampilkan riwayat transaksi rekening.